ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 13

# Sesiones, login y seguridad

- La taquilla del gimnasio: cookies y sesiones
- Sesiones con $_SESSION
- Cookies con setcookie()
- Contraseñas: password_hash y password_verify
- Un login completo en tres archivos
- Seguridad: los tres ataques que tenéis que conocer



**Ramón Abramo**

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

# 🧭 Contenido de la clase

Os voy a contar un secreto incómodo: **la web no tiene memoria**. Cada vez que hacéis clic en un enlace, para el servidor sois un perfecto desconocido, aunque hayáis entrado hace dos segundos. El protocolo HTTP funciona así: pregunta, respuesta y olvido.

¿Y entonces cómo sabe Amazon qué hay en vuestro carrito? ¿Cómo sabe el campus virtual que ya habéis iniciado sesión? Con **cookies** y **sesiones**. Hoy las vemos, montamos un **login** de verdad y cerramos la clase con lo imprescindible de **seguridad**.

**1**

🤔 **La taquilla del gimnasio: cookies y sesiones**



**2**

📘 **Sesiones con $_SESSION**



**3**

🔍 **Cookies con setcookie()**



**4**

⚠️ **Contraseñas: password_hash y password_verify**



**5**

👌 **Un login completo en tres archivos**



**6**

🐞 **Seguridad: los tres ataques que tenéis que conocer**



**7**

🚀 **Ejercicios**



**8**

👋 **Lo que tenéis que llevaros de hoy**





CLASE 13 · PUNTO 1

## 🤔 La taquilla del gimnasio: cookies y sesiones

Llegáis al gimnasio, os identificáis en recepción y el recepcionista guarda vuestras cosas en una **taquilla** del vestuario. A vosotros no os da la mochila: os da una **pulsera con un número**. Cada vez que pasáis por la puerta, enseñáis la pulsera y saben quiénes sois y cuál es vuestra taquilla.

| En el gimnasio | En la web |
|---|---|
| La taquilla, dentro del vestuario | La **sesión**: los datos se guardan **en el servidor** |
| La pulsera con el número | Una **cookie** llamada `PHPSESSID` con el identificador de la sesión |
| Enseñar la pulsera en la puerta | El navegador envía la cookie en **cada** petición, automáticamente |

Así que hay dos herramientas, y cada una tiene su uso [1]:

| | 🍪 Cookie | 🗄️ Sesión |
|---|---|---|
| Dónde se guarda | En el **navegador** del usuario | En el **servidor** |
| ¿Puede el usuario verla o cambiarla? | Sí, desde las herramientas del navegador | No: solo tiene el número de la pulsera |
| Cuánto dura | Lo que le digáis: días, meses... | Hasta cerrar el navegador (por defecto) |
| Úsala para... | Preferencias: tema oscuro, idioma | Usuario conectado, carrito, rol, permisos |



🎯 **Regla:** todo lo que sea **importante** (quién es el usuario, qué rol tiene, cuánto cuesta el carrito) va en la **sesión**. Si lo guardarais en una cookie, cualquiera podría cambiar `rol=alumno` por `rol=admin`.



CLASE 13 · PUNTO 2

## 📘 Sesiones con $_SESSION

Trabajar con sesiones es sorprendentemente sencillo:

1. Llamad a `session_start()` al principio de cada página que use la sesión.
2. Usad `$_SESSION` como un array normal: lo que guardéis ahí seguirá estando en la página siguiente.

01-contador-visitas.php — php-paso-a-paso/clase13 — Visual Studio Code

01-contador-visitas.php

```php
<?php
declare(strict_types=1);
session_start();

// Si es la primera visita, no existe: empezamos en 0
$_SESSION["visitas"] = ($_SESSION["visitas"] ?? 0) + 1;

echo "Has visitado esta página {$_SESSION['visitas']} veces";
```

📄 Ejemplo: `01-contador-visitas.php` . Recargad con F5 y veréis cómo sube el contador.



⚠️ `session_start()` necesita enviar una cookie, y las cookies viajan en las **cabeceras**. Por eso tiene que ir **antes de cualquier HTML o** `echo` . Si no, error *«headers already sent»*, el mismo que con `header()` en la clase 12.

## Un atajo nuevo: `??=`

En las soluciones veréis esto:

ejemplo.php — php-paso-a-paso/clase13 — Visual Studio Code

ejemplo.php

```php
<?php
$_SESSION["carrito"] ??= [];
```

Significa: «si `$_SESSION["carrito"]` no existe, créalo con un array vacío; si ya existe, déjalo como está». Es el primo del `??` de la clase 7.



CLASE 13 · PUNTO 3

## 🔍 Cookies con setcookie()

Una cookie es un pequeño texto que el servidor le pide al navegador que guarde. Perfectas para recordar **preferencias** [2]:

02-cookie-tema.php — php-paso-a-paso/clase13 — Visual Studio Code

02-cookie-tema.php

```php
<?php
setcookie("tema", "oscuro", [
    "expires" => time() + 60 * 60 * 24 * 30,    // dentro de 30 días
    "path" => "/",                              // válida en toda la web
    "httponly" => true,                         // JavaScript no puede leerla
    "samesite" => "Lax",                        // protección básica frente a otras webs
]);
```

Y para leerla, en las peticiones siguientes:

02-cookie-tema.php — php-paso-a-paso/clase13 — Visual Studio Code

02-cookie-tema.php

```php
<?php
$tema = $_COOKIE["tema"] ?? "claro";
```

| Opción | Para qué sirve |
|---|---|
| `expires` | Cuándo caduca. `time()` es «ahora» en segundos; le sumamos los segundos que queramos |
| `httponly` | Impide que un script malicioso (XSS) la robe. Ponedlo siempre |
| `secure` | Solo viaja por HTTPS. Activadlo cuando la web esté publicada con certificado |
| `samesite` | Limita que otras webs la envíen. `Lax` es un buen valor por defecto |



⚠️ Una cookie recién creada **no aparece en `$_COOKIE` hasta la siguiente petición**. Por eso en el ejemplo redirigimos justo después de crearla.

⚠️ **Nunca os fiéis de una cookie**: el usuario puede modificarla. Comprobad siempre que su valor es uno de los permitidos.

📄 Ejemplo: `02-cookie-tema.php` . Elegid el tema oscuro, cerrad el navegador y volved: lo recuerda.



CLASE 13 · PUNTO 4

## ⚠️ Contraseñas: password_hash y password_verify

Primera norma de las contraseñas: **nunca se guardan tal cual**. Si alguien roba vuestra base de datos (pasa más de lo que creéis), tendría las contraseñas de todos vuestros usuarios... que seguramente usan la misma en otros sitios.

Lo que se guarda es un **hash**: una especie de huella digital de la contraseña que **no se puede deshacer** [3].

03-generar-hash.php — php-paso-a-paso/clase13 — Visual Studio Code

03-generar-hash.php

```php
<?php
// Al REGISTRAR al usuario: guardamos el hash, no la contraseña
$hash = password_hash("secreto123", PASSWORD_DEFAULT);
// $2y$10$MPEZrmnuTGW2ABJ39YJP3.eYanNk5GE3PJzWAFDmxvReNpEt0xnoC

// Al INICIAR SESIÓN: comprobamos si la contraseña escrita encaja con el hash
if (password_verify($claveEscrita, $hash)) {
    echo "Contraseña correcta";
}
```

Lo curioso: si generáis el hash dos veces, **sale distinto cada vez** (PHP le añade un ingrediente aleatorio, la «sal»). Y aun así `password_verify()` sabe comprobarlo. No intentéis comparar hashes con `===`: usad siempre `password_verify()`.

📄 Ejemplo: `03-generar-hash.php` . Recargad varias veces y mirad cómo cambia el hash.



🚫 Veréis por internet `md5()` y `sha1()` para contraseñas. **Ni se os ocurra**: son algoritmos rápidos y viejos que hoy se rompen en muy poco tiempo. Para contraseñas, `password_hash()` y nada más.



CLASE 13 · PUNTO 5

## 👌 Un login completo en tres archivos

Un área privada necesita tres piezas: la **puerta** (login), el **guardián** (que protege cada página privada) y la **salida** (logout).

Esquema

```
login.php ──(usuario y contraseña correctos)──► panel.php ──(botón)──► logout.php
    ▲                                                │                      │
    └──────────── el guardián te echa si no ─────────┘                      │
    └──────────────────────────────────────────────────────────────────────┘
```

Los usuarios están en `usuarios.php`, un archivo que **devuelve** un array con `return`. Lo cargamos con `$usuarios = require __DIR__ . "/usuarios.php";`. En la clase 14 veréis `require` con calma, y más adelante los usuarios vendrán de una base de datos.

## 🚪 La puerta: `login.php`

ejemplo.php — php-paso-a-paso/clase13 — Visual Studio Code

ejemplo.php

```php
<?php
$error = "";

session_start();
$usuarios = require __DIR__ . "/usuarios.php";
$esEnvio = $_SERVER["REQUEST_METHOD"] === "POST";
$usuario = trim($_POST["usuario"] ?? "");
$clave = $_POST["clave"] ?? "";

if ($esEnvio) {
    $datos = $usuarios[$usuario] ?? null;

    if ($datos !== null && password_verify($clave, $datos["hash"])) {
        session_regenerate_id(true);           // pulsera nueva al entrar
        $_SESSION["usuario"] = $usuario;
        $_SESSION["rol"] = $datos["rol"];
        header("Location: panel.php");
        exit;
    }
    $error = "Usuario o contraseña incorrectos.";
}
```

Dos detalles de seguridad:

- `session_regenerate_id(true)` cambia el número de la «pulsera» al iniciar sesión. Así, si alguien hubiera conseguido el número antiguo, ya no le sirve.
- El mensaje de error es **el mismo** si falla el usuario o la contraseña. Si dijerais «este usuario no existe», le estaríais ayudando a quien intenta adivinar.

## 💂 El guardián: `panel.php`

Estas líneas van **al principio de cada página privada**:

ejemplo.php — php-paso-a-paso/clase13 — Visual Studio Code

ejemplo.php

```php
<?php
session_start();

if (!isset($_SESSION["usuario"])) {
    header("Location: login.php");
    exit;   // ¡IMPRESCINDIBLE!
}
```



⚠️ Sin el `exit;`, el navegador recibe la orden de irse… pero **PHP sigue ejecutando** el resto de la página en el servidor. Si debajo hubiera datos privados o un borrado, se ejecutaría igualmente.

## 🚶 La salida: `logout.php`

ejemplo.php — php-paso-a-paso/clase13 — Visual Studio Code

ejemplo.php

```php
<?php
session_start();
$_SESSION = [];        // vaciar la taquilla
session_destroy();     // y destruirla
header("Location: login.php?salir=1");
exit;
```

## ✅ Pruebas para comprobarlo

| Haced esto | Debéis obtener |
|---|---|
| Abrid `panel.php` directamente, sin iniciar sesión | Os manda a `login.php` |
| Entrad con `ana` y una contraseña incorrecta | «Usuario o contraseña incorrectos» |
| Entrad con `ana` / `secreto123` | El panel, con la zona de administrador |
| Entrad con `luis` / `secreto123` | El panel, **sin** la zona de administrador |
| Cerrad sesión y pulsad «Atrás» en el navegador | Al recargar, el guardián os echa |



CLASE 13 · PUNTO 6

## 🐞 Seguridad: los tres ataques que tenéis que conocer

En cuanto publiquéis una web, habrá programas automáticos probando vuestros formularios cada pocos minutos. No es paranoia: es el día a día de internet. Estos son los tres ataques más comunes, y la defensa de cada uno [4]:

| Ataque | Qué hace el atacante | Defensa |
|---|---|---|
| **XSS** | Mete código JavaScript en un campo para que se ejecute en el navegador de otros | `htmlspecialchars()` al mostrar (clase 12) y cookies `httponly` |
| **Inyección SQL** | Escribe trozos de SQL en un formulario para alterar vuestras consultas a la base de datos | **Consultas preparadas** con PDO (lo veremos a fondo con las bases de datos) |
| **CSRF** | Desde **otra web**, hace que vuestro navegador envíe un formulario a la vuestra aprovechando que tenéis la sesión abierta | Un **token secreto** en cada formulario |

## El token CSRF, paso a paso

Imaginad que estáis en vuestra tienda con la sesión abierta y, en otra pestaña, visitáis una web trampa. Esa web tiene un formulario invisible que envía «cambiar mi email a `ladron@...`» a vuestra tienda. Vuestro navegador manda la cookie de sesión con él... y la tienda se lo cree.

La defensa: cada formulario lleva un **código secreto** que solo conoce vuestra web. La web trampa no puede adivinarlo.

04-csrf.php — php-paso-a-paso/clase13 — Visual Studio Code

04-csrf.php

```php
<?php
session_start();

// 1. Crear el token (una vez por sesión) y guardarlo en la sesión
if (empty($_SESSION["csrf"])) {
    $_SESSION["csrf"] = bin2hex(random_bytes(32));
}

// 3. Al recibir el formulario: ¿coincide el token?
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    if (!hash_equals($_SESSION["csrf"], $_POST["csrf"] ?? "")) {
        http_response_code(403);
        exit("Petición no autorizada.");
    }
    // ...procesar con tranquilidad
}
?>
<form method="post">
    <!— 2. El token viaja escondido dentro del formulario —>
    <input type="hidden" name="csrf" value="<?= $_SESSION["csrf"] ?>">
    ...
</form>
```

`hash_equals()` compara dos textos de forma segura; para tokens, mejor que `===`.

📄 Ejemplo: `04-csrf.php`. Pulsad F12, cambiad el valor del campo oculto y enviad: veréis el error 403.



💡 ¿Os parece mucho trabajo para cada formulario? Lo es. Por eso los *frameworks* lo hacen solos: en Laravel bastará con escribir `@csrf` dentro del formulario. Pero ahora sabréis qué hace por dentro.



CLASE 13 · PUNTO 7

## 🚀 Ejercicios

- ☐ **01 · Carrito de la compra.** Tres productos con botón «Añadir». El carrito, guardado en la sesión, muestra las cantidades, el total y permite quitar productos y vaciarlo. → `ej01-carrito.php`
- ☐ **02 · Idioma.** Enlaces ES · EN · FR que guarden el idioma elegido en una cookie de un año y muestren la página en ese idioma. ¿Qué pasa si alguien pone a mano un idioma que no existe? → `ej02-idioma.php`
- ☐ **03 · Intentos de login.** Un login que, tras 3 intentos fallidos, se bloquee durante 60 segundos. → `ej03-intentos.php`



🏆 **Reto:** añadid al login de los ejemplos una página `admin.php` a la que **solo** puedan entrar los usuarios con rol `admin`. Los alumnos que lo intenten deben ver un error 403 (`http_response_code(403)`).



CLASE 13 · PUNTO 8

## 👋 Lo que tenéis que llevaros de hoy

| Idea | En una frase |
|---|---|
| La web no tiene memoria | Cookies y sesiones le dan memoria |
| Sesión | Datos en el servidor. `session_start()` arriba y `$_SESSION` como un array |
| Cookie | Datos en el navegador. Solo para preferencias; nunca os fiéis de ella |
| Contraseñas | `password_hash()` para guardar y `password_verify()` para comprobar. Nada de `md5()` |
| Login | `session_regenerate_id(true)` al entrar; mismo error para usuario y contraseña |
| Guardián | En cada página privada: sin sesión, `header("Location: ...")` y `exit;` |
| Logout | `$_SESSION = [];` y `session_destroy();` |
| XSS / SQL / CSRF | `htmlspecialchars()` / consultas preparadas / token en cada formulario |

# 📚 Fuentes y documentación oficial

1. Manual de PHP: manejo de sesiones
2. Manual de PHP: setcookie
3. Manual de PHP: hash de contraseñas
4. OWASP Top Ten